



LISTAS ENLAZADAS

Data Structures

Carlos Rojas-Sánchez

UNIVERSIDAD DEL MAR

1. Retroalimentación con apuntadores
2. Representación de una lista enlazada
3. Operaciones con listas enlazadas
4. Tipos de listas enlazadas
5. Aplicaciones de las listas enlazadas

Retroalimentación con apuntadores

¿Qué es un Apuntador?

Un **apuntador** (o puntero) es una variable que almacena la **dirección de memoria** de otra variable.

Conceptos Clave

- **Dirección de memoria:** Ubicación física en la RAM donde se guarda un dato.
- **Operador de dirección (&):** Obtiene la dirección de memoria de una variable.
- **Operador de indirección (*):** Accede al valor almacenado en la dirección a la que apunta el puntero.

Declaración e Inicialización

Sintaxis básica para declarar y vincular un puntero a una variable existente:

```
#include <stdio.h>

int main() {
    int x = 42;
    int *p = &x; // 'p' almacena la dirección de memoria de 'x'

    printf("Valor de x: %d\n", x);
    printf("Direccion de x (&x): %p\n", (void*)&x);
    printf("Valor de p (p):      %p\n", (void*)p);
    printf("Valor apuntado (*p): %d\n", *p);

    return 0;
}
```

Modificación por Indirección

Modificar el contenido de la memoria a través del apuntador altera directamente la variable original.

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr = &num;

    *ptr = 20; // Cambia el valor de 'num' a 20

    printf("Nuevo valor de num: %d\n", num); // Imprime 20
    return 0;
}
```

Punteros Especiales: NULL y Void

Puntero Nulo (NULL)

- Indica que el puntero **no apunta a ninguna ubicación válida** de memoria.
- Fundamental en listas enlazadas para indicar el final de la lista.
- Buena práctica: Inicializar siempre punteros no asignados con **NULL**.

Puntero Genérico (void*)

- Puede apuntar a datos de cualquier tipo.
- Usado por funciones de asignación de memoria como **malloc**.

Paso por Referencia en Funciones

En C, todo paso de parámetros es por valor. Para modificar variables dentro de una función, pasamos sus **direcciones**.

```
#include <stdio.h>

void intercambiar(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    intercambiar(&x, &y);
    printf("x: %d, y: %d\n", x, y); // x=10, y=5
    return 0;
}
```


Sumar o restar a un puntero lo desplaza en bloques equivalentes al **tamaño de su tipo de dato**.

```
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *p = arr; // Equivale a &arr[0]

    printf("Elemento 0: %d\n", *p);          // 10
    printf("Elemento 1: %d\n", *(p + 1));    // 20
    printf("Elemento 2: %d\n", *(p + 2));    // 30

    return 0;
}
```

Punteros a Estructuras

Para acceder a miembros de una estructura mediante un puntero, se usa el operador flecha (->).

```
#include <stdio.h>

typedef struct {
    int valor;
} Elemento;

int main() {
    Elemento e = {100};
    Elemento *ptr = &e;

    // Formas equivalentes de acceso:
    printf("Valor: %d\n", (*ptr).valor);
    printf("Valor: %d\n", ptr->valor); // Preferido

    return 0;
}
```

Punteros Dobles (T**)

Un puntero doble almacena la dirección de otro puntero.

Uso clave en Listas Enlazadas: Permite modificar la cabeza (*head*) de una lista desde dentro de una función auxiliar.

```
void cambiar_puntero(int **ptr_doble, int *nuevo_destino) {  
    *ptr_doble = nuevo_destino; // Cambia a dónde apunta el puntero original  
}
```

Atención a estos fallos típicos en C

1. **Desreferenciar `NULL`:** Intentar acceder a `*p` cuando `p == NULL` causa un *Segmentation Fault*.
2. **Punteros Colgantes (*Dangling Pointers*):** Punteros que apuntan a memoria que ya fue liberada con `free()`.
3. **Fugas de Memoria (*Memory Leaks*):** Perder la referencia a un bloque de memoria reservado en el **Heap** sin haberlo liberado.

Resumen: Punteros en el contexto de Listas

Concepto	Aplicación en Listas
<code>struct Nodo*</code>	Conecta un nodo con el siguiente en la secuencia.
<code>NULL</code>	Marca el fin de la lista enlazada.
<code>malloc(sizeof(Nodo))</code>	Crea dinámicamente nuevos nodos en el <i>Heap</i> .
<code>free(nodo)</code>	Elimina un nodo y libera la memoria.

Representación de una lista enlazada

¿Qué es una Lista Enlazada?

Es una estructura de datos **dinámica y lineal** formada por una colección de elementos llamados **nodos**.

Características Principales

- **No contigua en memoria:** A diferencia de un arreglo, los elementos no ocupan posiciones consecutivas en la RAM.
- **Tamaño dinámico:** Crece o se reduce en tiempo de ejecución según se necesite.
- **Enlaces explícitos:** Cada elemento almacena una referencia (puntero) hacia la ubicación del siguiente.

Estructura de un Nodo

El **nodo** es la unidad fundamental de almacenamiento en una lista enlazada.

Componentes de un Nodo

1. **Campo de Datos (*Data*):** Contiene el valor o la información a almacenar (enteros, cadenas, estructuras complejos, etc.).
2. **Campo Puntero (*Next*):** Puntero autorreferenciado que contiene la dirección de memoria del siguiente nodo de la lista.

[Dato | *Siguiete]

Definición del Nodo en C

Utilizamos una **estructura autorreferenciada** para representar el nodo:

```
#include <stdio.h>

// Definición de la estructura del Nodo
typedef struct Nodo {
    int dato;                // Información almacenada
    struct Nodo *siguiente; // Puntero al siguiente nodo
} Nodo;
```

Nota Importante

Es indispensable declarar **struct Nodo *siguiente;** dentro del **struct** antes de aplicar el alias con **typedef**.

Elementos de Control de la Lista

Para manipular y recorrer una lista enlazada requerimos dos elementos clave:

La Cabeza (*Head*)

- Puntero que apunta al **primer nodo** de la lista.
- Es el único punto de acceso directo a toda la estructura.
- Si la lista está vacía, **cabeza == NULL**.

El Fin de la Lista

- El puntero **siguiente** del **último nodo** apunta a **NULL**.
- Servirá como condición de parada al recorrer la lista.

Representación Gráfica de la Lista

Visualización de una lista enlazada simple con tres elementos:

Cabeza [0x100]



[10 | 0x200] → [20 | 0x300] → [30 | NULL]
(Dir: 0x100) (Dir: 0x200) (Dir: 0x300)

- La **cabeza** guarda la dirección inicial (**0x100**).
- Cada nodo apunta al bloque de memoria del siguiente.
- El último nodo apunta a **NULL** para indicar el fin.

Creación Manual de una Lista Simple

Ejemplo práctico de construcción de una lista de 2 nodos usando asignación en memoria dinámica:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 1. Crear el primer nodo
    Nodo *cabeza = (Nodo*)malloc(sizeof(Nodo));
    cabeza->dato = 10;

    // 2. Crear el segundo nodo
    Nodo *segundo = (Nodo*)malloc(sizeof(Nodo));
    segundo->dato = 20;

    // 3. Enlazar los nodos y finalizar con NULL
    cabeza->siguiente = segundo;
    segundo->siguiente = NULL;

    return 0;
}
```

Comparativa: Arreglo vs. Lista Enlazada

Arreglo (Array)	Lista Enlazada
Tamaño fijo en memoria.	Tamaño dinámico e inconstante.
Acceso directo $O(1)$ por índice.	Acceso secuencial $O(n)$.
Inserción/eliminación costosa.	Inserción/eliminación eficiente.
Memoria contigua.	Memoria dispersa en el <i>Heap</i> .

Operaciones con listas enlazadas

Operaciones Fundamentales

Las operaciones principales sobre una lista enlazada permiten gestionar sus nodos dinámicamente en memoria.

Categorías Principales

- **Inserción:** Agregar un nuevo nodo (al inicio, al final o en una posición intermedia).
- **Eliminación:** Remover un nodo existente liberando su memoria.
- **Recorrido y Búsqueda:** Transitar la lista nodo por nodo para consultar o procesar datos.

Principio Clave

A diferencia de los arreglos, estas operaciones no mueven los datos en memoria; solo reasignan los **punteros/enlaces**.

Paso a Paso: Inserción al Inicio

Para insertar un nodo al inicio de la lista se realizan 3 pasos secuenciales:

Algoritmo Visual

1. **Reserva de Memoria:** Crear el nuevo nodo en el *Heap* y asignar su dato.
2. **Enlace al frente:** Hacer que el campo **siguiente** del nuevo nodo apunte a la **cabeza** actual.
3. **Actualización de Cabeza:** Redirigir el puntero **cabeza** para que reconozca al nuevo nodo como el primero.

Estado Final:

Cabeza $\longrightarrow$ [Nuevo | *] $\longrightarrow$ [Viejo1 | *] $\longrightarrow$ [Viejo2 | NULL]

1. Código: Inserción al Inicio

Sintaxis completa en C empleando punteros dobles:

```
void insertar_inicio(Nodo **cabeza, int nuevo_dato) {  
    // 1. Reservar memoria para el nuevo nodo  
    Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));  
    nuevo->dato = nuevo_dato;  
  
    // 2. Apuntar el nuevo nodo a la cabeza actual  
    nuevo->siguiente = *cabeza;  
  
    // 3. Mover la cabeza para que apunte al nuevo nodo  
    *cabeza = nuevo;  
}
```

Complejidad Temporal: $\mathcal{O}(1)$ (Constante).

Para agregar un nodo al extremo posterior de la lista:

Algoritmo Visual

1. **Crear Nodo:** Asignar memoria e inicializar su puntero `siguiente = NULL`.
2. **Verificar Lista Vacía:** Si `cabeza == NULL`, el nuevo nodo pasa a ser la `cabeza`.
3. **Recorrer hasta el final:** Avanzar con un puntero auxiliar `actual` hasta que `actual->siguiente == NULL`.
4. **Reconectar:** Hacer `actual->siguiente = nuevo`.

2. Código: Inserción al Final

```
void insertar_final(Nodo **cabeza, int nuevo_dato) {
    Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));
    nuevo->dato = nuevo_dato;
    nuevo->siguiente = NULL;

    if (*cabeza == NULL) { // Si la lista está vacía
        *cabeza = nuevo;
        return;
    }

    Nodo *actual = *cabeza;
    while (actual->siguiente != NULL) { // Recorrer
        actual = actual->siguiente;
    }
    actual->siguiente = nuevo; // Enlazar al final
}
```

Complejidad Temporal: $\mathcal{O}(n)$ (Lineal).

Paso a Paso: Recorrido de la Lista

Para iterar e imprimir los valores de cada nodo:

Reglas de Control

1. Crear un puntero auxiliar (ej. `actual`) e igualarlo a `cabeza`.
2. Iterar mientras `actual != NULL`.
3. Leer/Procesar el campo `actual->dato`.
4. Avanzar al siguiente nodo haciendo `actual = actual->siguiente`.

Precaución

Jamás uses `cabeza = cabeza->siguiente` para recorrer la lista, ya que perderás el acceso a los nodos anteriores.

3. Código: Recorrido e Impresión

```
void imprimir_lista(Nodo *cabeza) {  
    Nodo *actual = cabeza;  
  
    printf("Lista: ");  
    while (actual != NULL) {  
        printf("[%d] -> ", actual->dato);  
        actual = actual->siguiente; // Avanzar  
    }  
    printf("NULL\n");  
}
```

4. Código: Búsqueda de un Elemento

Recorre la lista comparando una clave hasta encontrar coincidencia o llegar al final:

```
Nodo* buscar(Nodo *cabeza, int clave) {  
    Nodo *actual = cabeza;  
  
    while (actual != NULL) {  
        if (actual->dato == clave) {  
            return actual; // Elemento encontrado  
        }  
        actual = actual->siguiente;  
    }  
    return NULL; // Elemento no encontrado  
}
```

Complejidad Temporal: $\mathcal{O}(n)$ en el peor de los casos.

Paso a Paso: Eliminación del Primer Nodo

Pasos para remover la **cabeza** de forma segura sin dejar fugas de memoria:

Secuencia de Eliminación

1. ****Validar:**** Comprobar que la lista no esté vacía (`*cabeza != NULL`).
2. ****Guardar referencia:**** Asignar un puntero temporal `temp = *cabeza`.
3. ****Desvincular:**** Mover la cabeza al segundo nodo (`*cabeza = (*cabeza)->siguiente`).
4. ****Liberar:**** Invocar `free(temp)` para recuperar la memoria del antiguo primer nodo.

5. Código: Eliminación del Primer Nodo

```
void eliminar_inicio(Nodo **cabeza) {  
    if (*cabeza == NULL) return; // Lista vacía  
  
    Nodo *temp = *cabeza;          // Guardar referencia  
    *cabeza = (*cabeza)->siguiente; // Mover cabeza  
    free(temp);                    // Liberar memoria  
}
```

Complejidad Temporal: $\mathcal{O}(1)$.

Paso a Paso: Eliminación por Valor

Remover un nodo intermedio requiere actualizar la referencia de su antecesor:

Algoritmo de Reconexión

1. Mantener dos punteros durante la búsqueda: **actual** y **anterior**.
2. Si la clave está en el primer nodo, aplicar eliminación al inicio.
3. Si se encuentra en un nodo posterior, reconectar el antecesor omitiendo el objetivo:
`anterior->siguiente = actual->siguiente;`
4. Liberar la memoria con `free(actual)`.

6. Código: Eliminación por Valor

```
void eliminar_valor(Nodo **cabeza, int clave) {
    Nodo *actual = *cabeza, *anterior = NULL;

    // Caso 1: El nodo a eliminar es la cabeza
    if (actual != NULL && actual->dato == clave) {
        *cabeza = actual->siguiente;
        free(actual);
        return;
    }
    // Caso 2: Buscar guardando el nodo anterior
    while (actual != NULL && actual->dato != clave) {
        anterior = actual;
        actual = actual->siguiente;
    }
    if (actual == NULL) return; // No encontrado

    // Reconectar y liberar
    anterior->siguiente = actual->siguiente;
    free(actual);
}
```

Tipos de listas enlazadas

Clasificación de las Listas Enlazadas

Existen distintas variantes de listas enlazadas según la forma en que sus nodos se conectan entre sí.

Tipos Principales

- **Listas Simplemente Enlazadas:** Un solo enlace por nodo (navegación unidireccional).
- **Listas Doblemente Enlazadas:** Dos enlaces por nodo (navegación bidireccional).
- **Listas Circulares:** El último nodo se conecta de vuelta al primero (sin final **NULL**).

1. Listas Simplemente Enlazadas

Es la estructura básica donde cada nodo contiene solo un puntero al siguiente elemento.

Características

- **Navegación:** Exclusivamente hacia adelante (de izquierda a derecha).
- **Último enlace:** Apunta directamente a **NULL**.
- **Uso de memoria:** Menor consumo por nodo (solo almacena un puntero).

Cabeza $\longrightarrow$ [10 | *] $\longrightarrow$ [20 | *] $\longrightarrow$ [30 | NULL]

Estructura de Nodo: Lista Simple

Definición estándar en C para un nodo de lista simple:

```
typedef struct NodoSimple {  
    int dato;  
    struct NodoSimple *siguiente;  
} NodoSimple;
```

Ventajas y Desventajas

- **Ventaja:** Implementación sencilla y menor gasto de memoria RAM.
- **Desventaja:** Imposible retroceder al nodo anterior sin volver a empezar desde la cabeza.

2. Listas Doblemente Enlazadas

Cada nodo posee dos punteros: uno hacia el nodo **siguiente** y otro hacia el nodo **anterior**.

Características

- **Navegación:** Bidireccional (hacia adelante y hacia atrás).
- **Control de extremos:** Suele usarse una **Cabeza** (*Head*) y una **Cola** (*Tail*).
- **Eliminación eficiente:** Permite eliminar un nodo en $\mathcal{O}(1)$ si ya se tiene la referencia directa a él.

$\text{NULL} \longleftrightarrow [* | 10 | *] \longleftrightarrow [* | 20 | *] \longleftrightarrow [* | 30 | *] \longleftrightarrow \text{NULL}$

Estructura de Nodo: Lista Doble

Definición en C incluyendo el puntero al nodo anterior:

```
typedef struct NodoDoble {  
    int dato;  
    struct NodoDoble *anterior; // Puntero al nodo previo  
    struct NodoDoble *siguiente; // Puntero al nodo posterior  
} NodoDoble;
```

Atención al Insertar/Eliminar

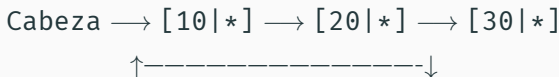
Requiere actualizar ****cuatro enlaces**** por operación para mantener la consistencia en ambas direcciones.

3. Listas Circulares

El campo **siguiente** del último nodo no apunta a **NULL**, sino de regreso al ****primer nodo**** (cabeza).

Variantes

- **Circular Simple:** Enlace único que vuelve al inicio desde el final.
- **Circular Doble:** El primer nodo apunta al último como su **anterior**, y el último apunta al primero como su **siguiente**.



Paso a Paso: Recorrido en Lista Circular

Al no existir un fin marcado por **NULL**, la condición de parada cambia:

Algoritmo de Recorrido

1. Guardar la dirección del nodo inicial (**cabeza**).
2. Usar una estructura **do-while** para procesar el nodo actual.
3. Avanzar al siguiente nodo (**actual = actual->siguiente**).
4. Detener el ciclo cuando **actual == cabeza**.

Riesgo de Bucle Infinito

Si la condición de parada falla o se evalúa con **while(actual != NULL)**, el programa entrará en un ciclo infinito.

Código: Recorrido de Lista Circular Simple

```
void imprimir_circular(NodoSimple *cabeza) {
    if (cabeza == NULL) return;

    NodoSimple *actual = cabeza;
    do {
        printf("[%d] -> ", actual->dato);
        actual = actual->siguiente;
    } while (actual != cabeza); // Condición de parada

    printf("(Cabeza)\n");
}
```

Cuadro Comparativo de Estructuras

Criterio	Simple	Doble	Circular
Enlaces por nodo	1 (sig)	2 (ant/sig)	1 o 2
Navegación	Adelante	Ambos senti- dos	En ciclo
Fin de lista	NULL	NULL	Regresa al ini- cio
Costo memoria	Bajo	Alto	Bajo / Alto
Casos de uso	Pilas, colas	Historiales, búferes	Round-robin

Aplicaciones de las listas enlazadas

Las listas enlazadas son el bloque de construcción fundamental para implementar otras estructuras abstractas de datos (TDA).

Estructuras Derivadas

- **Pilas (LIFO):** Inserción y eliminación eficientes en $\mathcal{O}(1)$ al inicio de la lista (operaciones **push** y **pop**).
- **Colas (FIFO):** Inserción al final y eliminación al inicio en $\mathcal{O}(1)$ usando referencias a la *Cabeza* y *Cola*.
- **Grafos:** Representación de listas de adyacencia para optimizar el uso de memoria en grafos dispersos.

Casos de Uso: Listas Simples

Ideales para escenarios donde la navegación es estrictamente secuencial e unidireccional.

Ejemplos Reales

- **Gestión de Memoria Dinámica:** Mantenimiento de la lista de bloques de memoria libres (*Free List*) en sistemas operativos.
- **Tabla de Dispersión (Hash Tables):** Resolución de colisiones mediante el método de encadenamiento (*Chaining*).
- **Historiales simples:** Registro secuencial de eventos o logs que solo requieren lectura en orden cronológico.

Utilizadas cuando se requiere navegación en ambos sentidos o manipulación bidireccional de elementos.

Ejemplos Reales

- **Navegadores Web:** Botones "Atrás" y "Adelante" para transitar por el historial de pestañas.
- **Visores de Imágenes / Editores:** Desplazamiento hacia la foto anterior o siguiente sin reescaneo.
- **Mecanismo Deshacer/Rehacer (Undo/Redo):** Gestión del historial de acciones en aplicaciones como procesadores de texto o software de diseño.

Casos de Uso: Listas Circulares

Indispensables en sistemas que procesan tareas en ciclos continuos e infinitos.

Ejemplos Reales

- **Planificación de Procesos (Round-Robin):** El SO asigna tiempos de CPU a múltiples hilos o procesos distribuidos en anillo.
- **Reproductores de Música:** Lista de reproducción en modo bucle (*repeat playlist*).
- **Búferes de Entrada/Salida:** Gestión de memoria circular para streaming de audio/video o datos transmitidos por red.

Ejemplo de Aplicación: Cola LIFO (Pila) con Lista Simple

Implementación sencilla de **push** utilizando la inserción al inicio:

```
typedef struct Nodo {
    int dato;
    struct Nodo *siguiente;
} Nodo;

// Operación Push (Insertar en la cima)
void push(Nodo **cima, int valor) {
    Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));
    nuevo->dato = valor;
    nuevo->siguiente = *cima;
    *cima = nuevo;
}
```

Ventaja: La pila puede crecer indefinidamente mientras exista memoria RAM disponible.

Resumen de Criterios de Selección

Requerimiento	Tipo Recomendado	Razonamiento
Ahorro estricto de memoria	Lista Simple	Solo utiliza 1 puntero por nodo.
Navegación bidireccional	Lista Doble	Permite usar puntero anterior .
Procesamiento continuo	Lista Circular	No requiere reiniciar punteros a NULL .
Inserción/Borrado rápido	Cualquier Lista	Operaciones en $\mathcal{O}(1)$ con puntero directo.